

Exercise 1: Implementing the Singleton Pattern

Main.java

```
public class Main {
    public static void main(String[] args) {

        System.out.println("Requesting Logger 1...");
        Logger logger1 = Logger.getInstance();
        logger1.logMessage("User logged in.");

        System.out.println("\nRequesting Logger 2...");
        Logger logger2 = Logger.getInstance();
        logger2.logMessage("Data saved to database.");

        System.out.println("\n--- Testing the Instances ---");
        if (logger1 == logger2) {
            System.out.println("SUCCESS: logger1 and logger2 are the exact same instance in memory!");
        } else {
            System.out.println("FAIL: They are different instances.");
        }
    }
}
```

Logger.java

```
public class Logger {

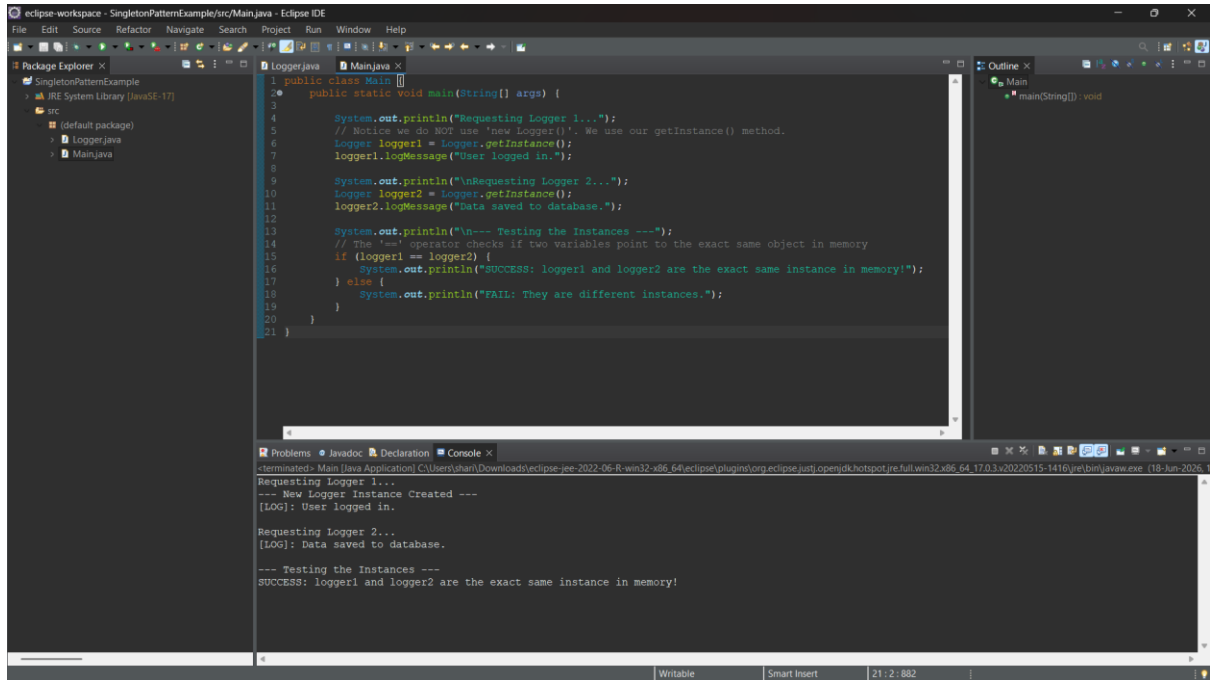
    private static Logger instance;

    private Logger() {
        System.out.println("--- New Logger Instance Created ---");
    }

    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }

    public void logMessage(String message) {
        System.out.println("[LOG]: " + message);
    }
}
```

OUTPUT



```
1 public class Main {
2     public static void main(String[] args) {
3
4         System.out.println("Requesting Logger 1...");
5         // Notice we do NOT use 'new Logger()'! We use our getInstance() method.
6         Logger logger1 = Logger.getInstance();
7         logger1.logMessage("User logged in.");
8
9         System.out.println("\nRequesting Logger 2...");
10        Logger logger2 = Logger.getInstance();
11        logger2.logMessage("Data saved to database.");
12
13        System.out.println("\n--- Testing the Instances ---");
14        // The '==' operator checks if two variables point to the exact same object in memory
15        if (logger1 == logger2) {
16            System.out.println("SUCCESS: logger1 and logger2 are the exact same instance in memory!");
17        } else {
18            System.out.println("FAIL: They are different instances.");
19        }
20    }
21 }
```

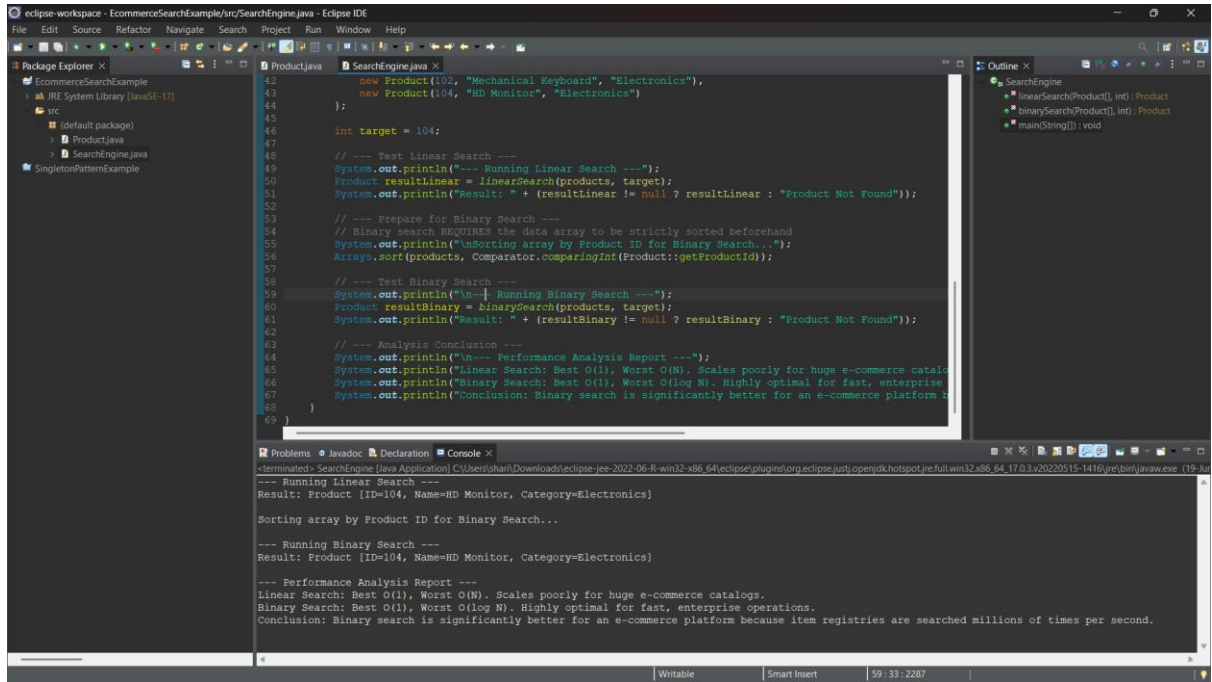
```
terminated- Main [Java Application] C:\Users\shan\Downloads\eclipse-jee-2022-06-R-win32-x86_64\eclipse\plugins\org.eclipse.jdt.openjdk.hotspot.jre.full.win32.x86_64.17.0.3.v20220515-1410\jre\bin\java.exe [18-Jun-2026 1
Requesting Logger 1...
--- New Logger Instance Created ---
[LOG]: User logged in.

Requesting Logger 2...
[LOG]: Data saved to database.

--- Testing the Instances ---
SUCCESS: logger1 and logger2 are the exact same instance in memory!
```

Exercise 2: Implementing the Factory Method Pattern

Output



The screenshot displays the Eclipse IDE interface. The Package Explorer on the left shows the project structure: `EcommerceSearchExample` with sub-packages `src` and `src` (default package). The `src` package contains `Product.java` and `SearchEngine.java`. The `src` (default package) contains `SingletonPatternExample`. The main editor shows `SearchEngine.java` with the following code:

```
42     new Product(102, "Mechanical Keyboard", "Electronics"),
43     new Product(104, "HD Monitor", "Electronics")
44 };
45
46 int target = 104;
47
48 // --- Test Linear Search ---
49 System.out.println("---- Running Linear Search ----");
50 Product resultLinear = linearSearch(products, target);
51 System.out.println("Result: " + (resultLinear != null ? resultLinear : "Product Not Found"));
52
53 // --- Prepare for Binary Search ---
54 // Binary search REQUIRES the data array to be strictly sorted beforehand
55 System.out.println("\nSorting array by Product ID for Binary Search...");
56 Arrays.sort(products, Comparator.comparingInt(Product::getProductId));
57
58 // --- Test Binary Search ---
59 System.out.println("\n---- Running Binary Search ----");
60 Product resultBinary = binarySearch(products, target);
61 System.out.println("Result: " + (resultBinary != null ? resultBinary : "Product Not Found"));
62
63 // --- Analysis Conclusion ---
64 System.out.println("\n--- Performance Analysis Report ---");
65 System.out.println("Linear Search: Best O(1), Worst O(N). Scales poorly for huge e-commerce catalogs.");
66 System.out.println("Binary Search: Best O(1), Worst O(log N). Highly optimal for fast, enterprise operations.");
67 System.out.println("Conclusion: Binary search is significantly better for an e-commerce platform because item registries are searched millions of times per second.");
68 }
69 }
```

The Console window at the bottom shows the output of the program:

```
terminated> SearchEngine [Java Application] C:\Users\shah\Downloads\eclipse-jee-2022-06-R-win32-x86_64\eclipse\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.17.0.3.v20220515-1416\jre\bin\java.exe (19-fur
---- Running Linear Search ----
Result: Product [ID=104, Name=HD Monitor, Category=Electronics]

Sorting array by Product ID for Binary Search...

---- Running Binary Search ----
Result: Product [ID=104, Name=HD Monitor, Category=Electronics]

--- Performance Analysis Report ---
Linear Search: Best O(1), Worst O(N). Scales poorly for huge e-commerce catalogs.
Binary Search: Best O(1), Worst O(log N). Highly optimal for fast, enterprise operations.
Conclusion: Binary search is significantly better for an e-commerce platform because item registries are searched millions of times per second.
```